

Jianliang Wu *Simon Fraser University, Burnaby, BC*
Ruoyu Wu *Google*
Dongyan Xu, Dave (Jing) Tian, Antonio Bianchi *Purdue University, IN*

Editor: Steven Y. Ko



SECURING BLUETOOTH ACROSS LAYERS: A Systematic Review of Protection Strategies

Named after the Viking King Harald Bluetooth, Bluetooth is the de facto standard for short-range wireless communications. The introduction of Bluetooth Low Energy (BLE) and Mesh protocols has further paved the way for its domination in the era of IoT. Meanwhile, attacks against Bluetooth, such as BlueBorne [1] and BLESAs [2], have been booming in the past few years, impacting billions of devices. Thus, in this paper, we summarize a systematization-of-knowledge paper [3] focusing on the diverse experimental methods in Bluetooth vulnerability identification and prevention to shed some light on the landscape of this research domain.

BLUETOOTH ARCHITECTURE

As Figure 1 shows, Bluetooth is comprised of a controller stack and a host stack. The code of these two components usually executes on different microcontrollers (MCU). The controller stack code runs on an MCU within a Bluetooth chip, while the host stack code usually runs on the main MCU together with the host OS (e.g., Linux). We divide the Bluetooth stack into three layers based on their functionalities and the location where the functionalities should be implemented according to the specification. The physical layer is located at the bottom of the Bluetooth stack and is responsible for the physical signal process. The firmware layer lies in the controller on top of the physical layer, establishing link-layer connections and transmitting and processing data between the physical layer and the host layer. The host layer represents all components in the host stack, including the protocols for data exchange on the host stack, such as the Logical Link Control and Adaptation Protocol, and components to finish specific tasks on the host stack, such as Bluetooth profiles.

PHYSICAL LAYER DEFENSE

Defenses at the physical layer leverage the physical features of a device's Bluetooth signal to detect or prevent signal injection and signal eavesdropping. In general, a defense has two stages: a fingerprinting stage and an identification stage. The defense first fingerprints authorized devices by learning their unique physical features, such as the clock skew and signal strength. Then, in the identification stage, the defense keeps collecting the physical features of the communicating devices and checking whether these physical features are consistent with the fingerprints of the authorized devices. If they are inconsistent, it means signals from unauthorized devices are detected. The defense can warn the user of potential signal injection attacks. In addition to signal injection, physical layer defenses

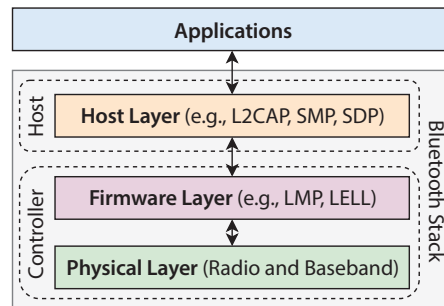


FIGURE 1. Layers of a Bluetooth stack.

can prevent signal eavesdropping attacks by selectively jamming the communication channel of authorized devices so that an attacker cannot receive the signal from them.

FIRMWARE LAYER DEFENSE

Like securing other protocols, at the implementation level, researchers have proposed approaches to fuzz and analyze the firmware (*firmware fuzzing and analysis*), through which implementation vulnerabilities can be revealed and further patched. At the design level, *formal verification*, an effective automatic method to detect design weaknesses, has also been adopted by researchers to either discover design weaknesses in the specification or provide security guarantees for the security protocols. Finally, to mitigate the issues caused by protocol design weaknesses, researchers have proposed enhancements (*protocol enhancement*) to fix the design weakness in the protocol.

Firmware Fuzzing and Analysis

Prior literature [4] has proven the effectiveness of firmware fuzzing and analysis in discovering firmware vulnerabilities. The same techniques are also applicable to Bluetooth firmware.

Fuzzing techniques can be adopted to discover memory corruption bugs in Bluetooth firmware. The fuzzing input could be generated from either a real device or an emulated device. In general, on the one hand, using a real device to provide fuzzing input

can be easily ported to fuzz the firmware of different controllers, while implementing the emulated device for different controllers' firmware needs extra effort. On the other hand, providing fuzzing input from an emulated device has better performance since the input is not transmitted Over The Air (OTA). Due to the closed-source nature of the firmware, firmware fuzzing requires either reverse-engineering the device's firmware (e.g., InternalBlue [5]) or self-implementing a piece of firmware (e.g., SweynTooth [6]).

Different from fuzzing, which is needed to run the firmware, statically analyzing the firmware can discover security or privacy issues caused by insecure configurations without running the firmware. For example, through static analysis, the insecure configuration that a BLE device can transmit data in plaintext allowing a passive attacker to eavesdrop on the data can be detected. In addition to directly discovering security issues, firmware analysis can help enhance firmware security, for example, by reducing the attack surface.

Formal Verification

While firmware fuzzing and analysis aim at automatically detecting issues in specific firmware implementations, formal verification attempts to detect design issues in the Bluetooth specification. Meanwhile, due to the development of automatic formal verification tools, such as ProVerif [7], automatic formal verification of Bluetooth protocols is feasible and efficient.

Since the Bluetooth pairing protocols are the cornerstones of Bluetooth security, they have drawn the most attention from researchers. Due to the complexity of pairing protocols, researchers have different focuses, such as a sub-protocol of a pairing method (the Diffie-Hellman protocol), when they model and analyze. Inspired by the attack exploiting two different pairing methods [8], researchers have also verified pairing protocols in a more comprehensive manner considering combinations of different pairing methods. Besides the pairing protocols, researchers have also verified the authentication and encryption procedures during the data exchange and identified issues that allow spoofing and illegal access attacks.

While automatic formal verification is effective in detecting design weaknesses, the

NO EXISTING DEFENSE TECHNIQUES CAN DEFEND AGAINST THE AUTHENTICATION BYPASS AND BREAKING ENCRYPTION ATTACKS

TABLE 1. Summarized defenses cross-checked with existing attack types. ✓: The defense can (partially) prevent the corresponding attack type.

Defense Types	Attack Types								
	Signal Eavesdropping	Signal Injection	Firmware Corruption	Break Pairing	Authentication Bypass	Break Encryption	Host Corruption	Illegal Access	Privacy Leakage
	Physical Layer		Firmware Layer				Host Layer		
Device Identification	✓	✓	–	–	–	–	–	✓	✓
Firmware Fuzzing and Analysis	–	–	✓	✓	–	–	–	–	✓
Formal Verification	–	–	–	✓	–	–	–	–	–
Protocol Enhancement	✓	–	–	–	–	–	–	–	–
Host Fuzzing and Analysis	–	–	–	–	–	–	✓	–	✓
Traffic Analysis and Filtering	–	–	–	–	–	–	✓	✓	–
System Hardening	–	–	–	–	–	–	–	✓	–

trade-off between the modeling granularity and coverage needs to be carefully considered by researchers to make the model computationally tractable. Finer-grained modeling may be able to discover more attacks, but it must sacrifice the model coverage to be tractable. On the contrary, for better model coverage, researchers may need to adopt a coarser-grained modeling choice for one part of the whole protocol to make the model tractable.

Protocol Enhancement

To address certain design weaknesses in Bluetooth protocols, different enhancements to these protocols have been proposed. Existing enhancements focus on two areas: BLE advertising and pairing.

To improve the security of BLE advertising, a three-way handshake scheme [9] was proposed to enhance BLE advertising and defend against replay attacks. Encryption was proposed and later adopted in the Bluetooth specification to provide confidentiality to the advertising messages. Apple implemented MagicPairing [10] to enhance the pairing protocol between Apple devices. It relies on the iCloud service to synchronize keys, based on which the Bluetooth secret key is derived and refreshed. By doing this, MagicPairing can achieve forward secrecy that the specification does not provide.

Though enhancing the protocol can address its design weaknesses fundamentally, these enhancements face compatibility issues since they require modifications of devices on both ends of the communication. For this reason, to the best of our knowledge, the three-way handshake scheme is not adopted by real-world devices, and MagicPairing is enabled only among Apple products.

HOST LAYER DEFENSE

The firmware layer and the host layer share some defense techniques because they are both pieces of software essentially. The host code can also be secured via fuzzing and static analysis (*host fuzzing and analysis*), and protocols at the host layer can also be enhanced (*system hardening*). Different from the enhancements at the firmware layer, which require protocol modifications introducing compatibility issues, enhancements at the host layer do not need protocol changes and usually can be deployed at one end of the communication.

There are also different defense techniques between the firmware layer and the host layer. Potentially, formal verification is also applicable at the host layer. However, none of the existing formal verification can be categorized into the host layer because the target protocols of existing formal analyses are implemented in the firmware. Additionally, traffic analysis and filtering, the technique employed in securing other network protocols, has been applied to Bluetooth at the host layer while not applied at the firmware layer. The reason might be the closed-source nature of the firmware, which makes implementing traffic analysis mechanisms in the firmware challenging. Additionally, the limited computing power of the MCU that the firmware runs on may also impede the deployment of traffic analysis defenses.

Host Fuzzing and Analysis

Like firmware fuzzing, the fuzzing input for the host stack can also come from a real device OTA or from an emulated device. Similarly, fuzzing OTA provides better scalability but lower performance. It is noteworthy that existing fuzzing works are

either protocol-specific (e.g., SweynTooth only supports BLE) or platform-specific (e.g., Frankenstein [11] only supports Linux). A generic fuzzing tool for the host layer is still missing. In terms of host code analysis, instead of analyzing the host code itself, a misconfigured device can be identified by analyzing its communicating counterpart, e.g., a device's mobile companion app, which is different from the firmware analysis.

System Hardening

Defenses in this subcategory focus on improving Bluetooth security by enhancing its implementations. The defenses either add extra code (e.g., access control for available services and additional security mechanisms, such as encryption and authentication at the application level) or remove unused code to harden the host code. Compared to the protocol enhancements, hardening the host code does not change the protocol, and thus introduces little compatibility issues. For example, introducing extra access control policies [12] or removing unused code [13] only needs to modify the high-end device (e.g., Android phones) and does not affect the benign connections. Adding additional security mechanisms at the application level requires modifications on both ends. However, since a device and its communicating counterpart (e.g., a companion app) are usually developed by the same vendor, the vendor can update the device and the app at the same time to minimize compatibility issues.

Traffic Analysis and Filtering

Like other protocols, the traffic analysis can be performed at the network gateway or a communication endpoint. Bluetooth traffic

can also be analyzed on a “gateway” device that sniffs the Bluetooth traffic from around devices, or on an endpoint device that communicates with other devices.

On the one hand, analyzing the traffic on a “gateway” device does not require modifications on the devices to be protected, but it needs extra hardware and faces the challenges introduced by Bluetooth frequency hopping. On the other hand, while analyzing and filtering Bluetooth traffic on an endpoint does not need extra hardware and is not affected by frequency hopping, it requires software modifications on the protected device. As such, performing traffic analysis on an endpoint may not be applicable to low-end devices whose software cannot be modified.

SUMMARY

Table 1 summarizes the defenses we surveyed against the attacks impacting Bluetooth, based on which, we draw the following insights.

Bluetooth fuzzing tools are effective yet not comprehensive

Due to the stateful nature of Bluetooth, stateful fuzzing is effective to discover implementation bugs at both the firmware layer and the host layer. However, compared to the firmware layer, which is well-supported, existing tools have limited support for the host layer. For example, SweynTooth can only fuzz the central role of BLE at the host layer. Because the state machines used to guide fuzzing in these tools are built manually and protocol-specific, existing tools cannot be readily adopted to fuzz other host protocols and profiles. A fuzzing tool that can fuzz the host layer (e.g., the host implementation on Linux and Android) is still missing.

Accordingly, one future research direction is to build fuzzing tools that support all protocols and profiles implemented at the host layer. Ideally, stateful fuzzing will be preferred, based on the effectiveness of existing stateful fuzzing tools at the firmware layer. A possible challenge is how to build a state machine to guide fuzzing across different protocols given that every protocol has its own state machine.

Certain defenses are still missing

As Table 1 shows, no existing defense techniques can defend against the authentication bypass and breaking encryption attacks. Consequently, the only approach to

protect a device from these attacks is to update its software and/or hardware to comply with a newer version of the Bluetooth specification (5.2 and above). However, updating software is already challenging, not to mention hardware updates. As such, most of the devices that are vulnerable to these attacks, such as KNOB [14], are still affected by these attacks.

The closed-source nature and the absence of patching mechanisms of the firmware limit the mitigation of these attacks. For instance, the pairing procedure of BLE is implemented in the host while the pairing procedure of Bluetooth Classic is in the firmware. Therefore, developers can fix the KNOB vulnerability of BLE by modifying the host code. For Bluetooth Classic, developers can only implement workarounds at the host layer since the firmware is closed-source and usually unmodifiable. One possible direction to address this issue is moving some firmware functionalities to the host to make patching easier. ■

Jianliang Wu is an Assistant Professor in the Computing Science Department at Simon Fraser University. His research investigates systems security problems at both the design and implementation levels on diverse computing platforms, including mobile and IoT. He received his PhD in Computer Science from Purdue University.

Ruoyu Wu is a Software Engineer at Google. His research involves applying program analysis techniques to secure or reverse engineer software related to machine learning and the Internet of Things. He received his PhD in Computer Science from Purdue University.

Dongyan Xu is a Samuel Conte Professor of Computer Science at Purdue University and Director of CERIAS, Purdue's cybersecurity research center. His research focuses on cyber and cyber-physical security. He received his PhD in Computer Science from the University of Illinois at Urbana-Champaign.

Dave (Jing) Tian is an Assistant Professor in the Department of Computer Science at Purdue University and faculty at the PurSec Lab. His research involves embedded systems security, operating systems security, trusted & confidential computing, and hardware security. He received his PhD from the Department of Computer & Information Science & Engineering at the University of Florida.

Antonio Bianchi is an Assistant Professor in the Computer Science Department at Purdue University and co-founder of the PurSec Lab. His research interests span the domains of software and systems security with a focus on enhancing the security of edge devices. He received his PhD in Computer Science from the University of California, Santa Barbara.

REFERENCES

- [1] Armis, “BlueBorne Technical White Paper,” <https://www.armis.com/research/blueborne/2020>.
- [2] J. Wu, Y. Nan, V. Kumar, D. J. Tian, A. Bianchi, M. Payer, and D. Xu. 2020. BLESAs: Spoofing attacks against reconnections in bluetooth low energy. *Proceedings of the USENIX Workshop on Offensive Technologies*.
- [3] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi. 2024. SoK: The long journey of exploiting and defending the legacy of King Harald bluetooth. *Proceedings of the IEEE Symposium on Security and Privacy*.
- [4] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti. 2014. A large-scale analysis of the security of embedded firmwares. 2014. *Proceedings of the USENIX Security Symposium*.
- [5] D. Mantz, J. Classen, M. Schulz, and M. Hollick. 2019. InternalBlue – bluetooth binary patching and experimentation framework. *Proceedings of the International Conference on Mobile Systems, Applications, and Services*.
- [6] M.E. Garbelini, C. Wang, S. Chattopadhyay, S. Sumei, and E. Kurniawan. 2020. SweynTooth: Unleashing mayhem over bluetooth low energy. *Proceedings of the USENIX Annual Technical Conference*.
- [7] B. Blanchet. 2016. Modeling and verifying security protocols with the applied pi calculus and ProVerif, *Foundations and Trends in Privacy and Security*, vol. 1.
- [8] M. von Tschirschnitz, L. Peuckert, F. Franzen, and J. Grossklags. 2021. Method confusion attack on bluetooth pairing. *Proceedings of the IEEE Symposium on Security and Privacy*.
- [9] P. Wang. 2014. “Bluetooth Low Energy-privacy Enhancement for Advertisement.” Master's thesis, Norwegian University of Science and Technology.
- [10] D. Heinze, J. Classen, and F. Rohrbach. 2020. MagicPairing: Apple's take on securing bluetooth peripherals. *Proceedings of the ACM Conference on Security and Privacy in Wireless and Mobile Networks*.
- [11] J. Ruge, J. Classen, F. Gringoli, and M. Hollick. 2020. Frankenstein: Advanced wireless fuzzing to exploit new bluetooth escalation targets. *Proceedings of the USENIX Security Symposium*.
- [12] M. Naveed, X.-y. Zhou, S. Demetriou, X. Wang, and C.A. Gunter. 2014. Inside job: Understanding and mitigating the threat of external device mis-binding on Android. *Proceedings of the Network and Distributed System Security Symposium*.
- [13] J. Wu, R. Wu, D. Antonioli, M. Payer, N. O. Tippenhauer, D. Xu, D. J. Tian, and A. Bianchi. 2021. LIGHTBLUE: Automatic profile-aware debloating of bluetooth stacks. *Proceedings of the USENIX Security Symposium*.
- [14] D. Antonioli, N.O. Tippenhauer, and K. Rasmussen. 2019. The KNOB is broken: Exploiting low entropy in the encryption key negotiation of bluetooth BR/EDR. *Proceedings of the USENIX Security Symposium*.